共3题，每题100分，考试时间3小时。

第一题

题目描述

给定一个字符串s和一个整数k，从字符串开头算起，每计数至$2k$个字符，就反转这$2k$字符中的前k个字符。

如果剩余字符少于k个，则将剩余字符全部反转。

如果剩余字符小于$2k$但大于或等于k个，则反转前k个字符，其余字符保持原样。

输入格式

输入为1行，由一个纯字母组成的字符串和一个非负整数组成，字符串和整数之间用空格分隔开。

输出格式

输出为1行，只需要输出反转后的字符串即可

样例 #1

样例输入

```
abcdefg 2
```

样例输出

```
bacdfeg
```

样例 #2

样例输入

```
abcd 2
```

样例输出

```
bacd
```

题解

这个问题等价于将字符串`s`分成k段，最后不足k的部分也计为一段，然后以每一段作为下标(从0开始)，第奇数段(即`(i+1)%!=0`或者`(i+1)&1`）的字符串进行反转，注意最后不足k的部分可以通过一个取小函数来实现。

```
#include<bits/stdc++.h>
using namespace std;

```

```
int main(){
  string s;
  int k;
  cin>>s>>k;
  int len=(int)s.size();
  int num=len/k;
  for(int i=0;i<num;i++){
    if((i+1)&1) reverse(s.begin()+i*k,min(s.begin()+i*k+k,s.end()));
  }
  cout<<s;
  return 0;
}
```

第二题

题目描述

毛哥买了一些糖果`candies`，打算把它们分给排好队的`n = num_people`个小朋友。

给第一个小朋友1颗糖果，第二个小朋友2颗，依此类推，直到给最后一个小朋友n颗糖果。

然后，我们再回到队伍的起点，给第一个小朋友$n+1$颗糖果，第二个小朋友$n+2$颗，依此类推，直到给最后一个小朋友$2*n$颗糖果。

重复上述过程（每次都比上一次多给出一颗糖果，当到达队伍终点后再次从队伍起点开始），直到我们分完所有的糖果。注意，就算我们手中的剩下糖果数不够（不比前一次发出的糖果多），这些糖果也会全部发给当前的小朋友。

返回一个长度为`num_people`、元素之和为`candies`的数组（我们用`ans`来表示），以表示糖果的最终分发情况（即`ans[i]`表示第i个小朋友分到的糖果数）。

输入格式

输入为1行，两个非负整数糖果数量（`candies`） 和 小朋友数量（`num_people`），他们之间用空格分隔开

输出格式

输出为1行，`num_people`个整数，中间用空格分开。每个整数代表小朋友分到的糖果数。

样例 #1

样例输入

```
7 4
```

样例输出

```
1 2 3 1
```

样例解释

第一次，`ans[0] += 1`，数组变为`[1,0,0,0]`。

第二次，`ans[1] += 2`，数组变为`[1,2,0,0]`。

第三次，`ans[2] += 3`，数组变为`[1,2,3,0]`。

第四次，`ans[3] += 1`（因为此时只剩下 1 颗糖果），最终数组变为`[1,2,3,1]`。

样例 #2

样例输入

```
10 3
```

样例输出

```
5 2 3
```

样例解释

第一次，`ans[0] += 1`，数组变为`[1,0,0]`。

第二次，`ans[1] += 2`，数组变为`[1,2,0]`。

第三次，`ans[2] += 3`，数组变为`[1,2,3]`。

第四次，`ans[0] += 4`，最终数组变为`[5,2,3]`。

数据范围

$1 \le$ `candies` $\le 10^9$

$1 \le$ `num_people` ≤ 1000

题解

依据题意模拟即可

```
#include<bits/stdc++.h>
using namespace std;

int main(){
  int candies,num_people;
  cin>>candies>>num_people;
  vector<int> ans(num_people,0);
  int i=0;
  int cnt=1;
  int sum=0;
  while(sum+cnt<=candies){
    ans[i%num_people]+=cnt;
    sum+=cnt;
    cnt++;
```

```
    i++;
  }
  ans[i%num_people]+=candies-sum;
  for(int x:ans) cout<<x<<" ";
  return 0;
}
```

第三题

题目描述

给你一个由 '1'（陆地）和 '0'（水）组成的的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

输入格式

输入为$n+1$行

第一行为两个用空格隔开的整数n和m

第1 - $n+1$行为$n \times m$矩阵，其中每行有m个用空格隔开的非负整数。

输出格式

输出为一个整数，即岛屿数量。

样例 #1

样例输入

```
4 5
0 0 1 1 0
1 1 0 1 0
1 1 0 0 0
0 0 0 0 0
```

样例输出

```
2
```

样例 #2

样例输入

```
4 5
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

样例输出

```
2
```

样例解释

没有'0'（水），全都是'1'（陆地），也按一个岛屿计算。

数据范围

$1 \le m, n \le 200$

题解

搜索板子题。

使用bfs的思想：枚举每一个位置的元素，如果是0就跳过，如果是1就使用bfs查询与该位置相邻的4个元素（前提是不出界），判断是否为1，如果是1那么继续查询，直到整个1块访问完毕。而为了避免走回头路可以定义一个bool数组inq(即in queue的缩写)来记录每个位置是否在bfs中已经入过队。

> **Tip**
>
> 对于当前位置(x,y)而言，可以设置两个增量数组来访问相邻的4个位置。
>
> ```
> int X[]={0,0,1,-1};
> int Y[]={1,-1,0,0};
> ```
>
> 分别对应(x,y+1) (x,y-1) (x+1,y) (x-1,y)
>
> 这样就可以用for循环去枚举4个方向，以确定当前坐标(nowX,nowY)相邻的4个位置
>
> ```
> for(int i=0;i<4;i++){
> newX=nowX+X[i];
> newY=nowY+Y[i];
> }
> ```

```
//BFS做法
#include<iostream>
#include<queue>
#include<algorithm>
using namespace std;
const int maxn=105;
struct node{
    int x,y; //存放位置
}Node;
int n,m; //矩阵大小
int matrix[maxn][maxn]; //01矩阵
bool inq[maxn][maxn]={false}; //记录是否入过队
int X[4]={0,0,1,-1},Y[4]={1,-1,0,0}; //坐标增量数组
bool judge(int x,int y){
    if(x>m||x<1||y>n||y<1||inq[x][y]||matrix[x][y]==0) return false;
    //只要出界||入过队||在该点不为1,就不访问
    return true;
```

```
}
//访问位置(x,y)所在的块，设置一个块内的1的inq为true
void bfs(int x,int y){
    queue<node> q;
    Node.x=x;Node.y=y;
    q.push(Node);
    while(!q.empty()){
        node top=q.front(); //取队首元素top
        q.pop();
        for(int i=0;i<4;i++){ //得到相邻位置
            int newX=top.x+X[i],newY=top.y+Y[i];
            if(judge(newX,newY)){ //如果这个新位置是符合要求的，需要访问
                Node.x=newX;Node.y=newY;
                q.push(Node); //入队
                inq[newX][newY]=true; //已经入过队
            }
        }
    }
}
int main(){
    cin>>m>>n;
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            cin>>matrix[i][j];
        }
    }
    int ans=0; //记录答案（块数
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            if(matrix[i][j]==1&&!inq[i][j]){ //如果这个位置是1而且没有被访问过
                ans++; //块数++
                bfs(i,j);
            }
        }
    }
    cout<<ans;
}
```

也可以使用DFS来实现。

```
#include<iostream>
#include<algorithm>
using namespace std;
/*dfs的思路:
从输入的点开始搜索，如果碰到0了就return
否则走岔路口：往四个方向走*/
const int maxn=105;
int m,n,matrix[maxn][maxn];
int X[4]={0,0,1,-1},Y[4]={1,-1,0,0}; //坐标增量数组
bool visited[maxn][maxn];//记录是否查询过
bool judge(int x,int y){
    if(x<1||x>m||y<1||y>n||visited[x][y]||!matrix[x][y]) return false;
    //只要出界||入过队||在该点不为1,就不访问
    return true;
}
```

```
//搜索一个块内，并标记为true
void dfs(int x,int y){
    if(!judge(x,y)) return;
    visited[x][y]=true;
    for(int i=0;i<4;i++){
        int newX=x+X[i],newY=y+Y[i];
        dfs(newX,newY);
    }
}
int main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            cin>>matrix[i][j];
        }
    }
    int ans=0; //记录答案（块数
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            if(matrix[i][j]&&!visited[i][j]){ //如果这个位置是1而且没有被访问过
                dfs(i,j);
                ans++;
            }
        }
    }
    cout<<ans;
}
```